



OOPs Problem Bank (Java)

Legend

★ = Core Placement Problem (Mandatory)

These problems should be completed by every student before moving to Exception Handling.

Section 1: Classes and Objects Basics

★ Create a Simple Class and Object

★ Class with Parameterized Constructor

★ Create Multiple Objects of a Class

Class with Default Field Values

★ Instance Variables vs Static (Class) Variables

Object Counter using a Static Variable

Class with Multiple Methods

Section 2: Methods and the this Keyword

★ Instance Method to Display Object Details

★ Method to Update Instance Variables using this

Method that Returns a Calculated Value

★ Class with Methods Calling Other Methods

★ Method Overloading (Compile-Time Polymorphism)

★ Static Method vs Instance Method

Constructor Chaining within the Same Class using this()

Section 3: Constructors and Object Lifecycle

★ Default (No-Argument) Constructor

★ Parameterized Constructor

★ Constructor Overloading

Constructor Chaining using this()

Instance Initializer Block

Garbage Collection Awareness (finalize()) as a Legacy Concept

Section 4: Encapsulation and Access Modifiers

★ Private Fields with Getter and Setter Methods

★ Public, Private, Protected, and Default (Package-Private) Access Demonstration

★ Bank Account Class with Encapsulated Balance



★ **Immutable Class using final Fields**

Validate Data using Setter Methods

Section 5: The final Keyword (Java-Specific)

★ **final Variable (Constant) Declaration**

★ **final Method (Preventing Override)**

★ **final Class (Preventing Inheritance)**

Demonstrate Compile-Time Error on Reassigning a final Variable

Section 6: Inheritance

★ **Single Inheritance — Parent and Child Class**

★ **Method Overriding in Child Class**

★ **Using super() to Call the Parent Constructor**

★ **Using super to Call a Parent Class Method**

★ **Multilevel Inheritance (Grandparent-Parent-Child)**

Hierarchical Inheritance (One Parent, Multiple Children)

★ **Why Java Does Not Support Multiple Inheritance of Classes**

Section 7: Polymorphism

★ **Method Overriding for Runtime Polymorphism**

★ **Method Overloading for Compile-Time Polymorphism**

★ **Upcasting and Downcasting Demonstration**

Dynamic Method Dispatch Example

Polymorphism using an Array of Parent Type References

Section 8: Abstraction

★ **Abstract Class using the abstract Keyword**

★ **Abstract Method Implementation in Child Classes**

Abstract Class Preventing Direct Object Creation

★ **Abstract Class with Both Abstract and Concrete Methods**

Section 9: Interfaces (Java-Specific)

★ **Create and Implement a Simple Interface**

★ **Implement Multiple Interfaces in a Single Class**

★ **Interface with a default Method**

Interface with a static Method

★ **Abstract Class vs Interface — When to Use Which**



★ **Functional Interface Basics (Single Abstract Method)**

Interface Inheritance (Interface Extending Interface)

Section 10: Object Class Overrides

★ **Override toString() for Readable Object Printing**

★ **Override equals() for Logical Object Comparison**

★ **Override hashCode() Alongside equals()**

★ **Override compareTo() using the Comparable Interface**

Implement Custom Ordering using the Comparator Interface

Section 11: Class Relationships

★ **Aggregation — Class Containing Another Class Object**

★ **Composition — Object Created and Owned by Another Class**

Class Containing a List of Objects

Many-to-One Relationship Modeling (Students and School)

Section 12: Enums as an OOP Construct (Java-Specific)

★ **Create a Basic Enum**

★ **Enum with Fields and a Constructor**

Enum with Methods

★ **Enum Used in a Switch Statement**

Section 13: Real World OOP Applications

★ **Student Management System using Classes**

★ **Employee Management System using Classes**

Library Management System (Books and Members)

★ **Bank Account System with Deposit/Withdraw Methods**

Shape Area Calculator using Inheritance and Interfaces

★ **Vehicle Rental System using OOP**

E-Commerce Product and Cart System using Classes

★ **Inventory Management System using OOP**